Portfolio Part 3 – Finalisation Abstract

Project links

- **AWS-hosted project website (static explainer):**

https://d2quyljtx3dqoe.cloudfront.net/

- **GitHub repository:**

https://github.com/EunatiaN1/multi-region-active-passive-failover

Project, outcome, and design

This portfolio project combines an interactive website about multi-region active-passive failover with infrastructure as code. The website explains the proposed AWS architecture and lets readers explore normal and simulated failure paths. Its controls are client-side illustrations: they do not invoke AWS APIs, change DNS, or demonstrate an actual recovery event.

The currently live website hosts that explainer as a static page. Its Terraform deployment provisions an S3 bucket with public access blocked, default encryption, versioning, and a lifecycle rule for non-current versions. CloudFront uses Origin Access Control (OAC) to read the object, redirects viewers to HTTPS, and applies the AWS-managed CachingOptimized cache policy. This deployed static-site root is deliberately independent of the new multi-region application example.

The submitted architecture proposes a primary application in eu-central-1 with a public Application Load Balancer across two Availability Zones and an EC2 Auto Scaling group. CloudFront forwards normal requests to the load balancer. A secondary S3 website in us-east-1 holds a maintenance page; CloudFront can request it when the primary origin returns HTTP 500, 502, 503, or 504. The original diagram also shows Route 53 health checks and DNS failover. These are separate mechanisms: CloudFront origin failover is response-code-driven, while DNS failover requires managed records and health checks for a controlled hostname.

The separate multi-region Terraform variant uses CloudFront's AWS-generated hostname so it can be inspected without a custom domain. It defines the primary VPC, ALB, EC2 Auto Scaling application, secondary S3 maintenance website, CloudFront origin group, and CloudWatch/SNS target-health alerting. It deliberately omits Route 53 records, DNS health checks, and ACM certificate provisioning. It is therefore an infrastructure-code prototype for origin failover, not a complete end-to-end DNS-failover service. It does not replace the existing live static website.

Making of, validation, and reflection

The Terraform application tier uses an Amazon Linux launch template and a small Python HTTP service managed by systemd. The threaded service handles concurrent requests, returns 404 for unknown paths, and runs as a dedicated unprivileged user with a restricted capability set and systemd filesystem protections. The target group checks `/health`; a distinct `/origin-health` CloudFront path is available for manual checking and does not drive Route 53. The Auto Scaling defaults are two minimum and desired instances, up to six, with target CPU utilization of 50 percent. Instance metadata requires IMDSv2 tokens. The ALB allows origin traffic from CloudFront's managed origin-facing prefix list, and instances accept HTTP only from the ALB security group.

The secondary website bucket uses a generated prefix and permits anonymous reads only for its maintenance `index.html`. The website endpoint itself is HTTP-only and directly reachable, so it must contain no confidential information. CloudFront can deliver a selected failover response through its HTTPS edge endpoint. The CloudWatch alarm observes unhealthy ALB targets and publishes to an SNS topic; an optional email recipient must confirm the subscription.

The project history records Terraform 1.12.2 formatting and provider-schema validation as passing, as well as eight automated, plan-only Terraform tests using a mocked AWS provider. The tests cover default capacity, invalid Auto Scaling capacity order, target health checks, IMDSv2, ALB and instance security-group boundaries, CloudFront failover response codes, maintenance-bucket public-access settings, alarm wiring, optional email subscription, project-name and region validation, and valid VPC subnet sizing. They require no AWS credentials and create no resources; they do not verify AWS permissions, deployment, or service behavior. Terraform was not available in the environment used to regenerate this submission, so the recorded checks were not rerun during this revision. The multi-region stack has not been applied or runtime-tested. The Phase 2 submission reports an earlier deployment and tests, but its corresponding source, state, and logs were not available to reproduce those results against this configuration. No live failover, scaling, alarm, or recovery result is claimed here.

Finalisation improved the application bootstrap, constrained network access, added input validation and mock-plan tests, and aligned this report with the configuration actually present. The remaining gaps are material: the multi-region stack is not deployed; CloudFront origin failover, alarms, scaling, and recovery have not been observed in AWS; and the Route 53 DNS layer is not implemented. Completing DNS failover requires a controlled domain and hosted zone, an explicit primary/standby DNS design, HTTPS certificate and endpoint decisions, tested health-check behavior, and account-specific cost and recovery review. A private or console S3 URL is not a domain or hosted-zone identifier. The validation register identifies the current evidence and the steps still needed. No tutor feedback was supplied for this revision; authentic feedback should be added if received.

Keywords: AWS, Terraform, Amazon S3, CloudFront, Application Load Balancer, Auto Scaling, active-passive, origin failover.